

PIShield: Securing LLM-Powered Code Assistants Against Prompt Injection Attacks Using Machine Learning

Mohammed Alghaith¹, Muhannad Almahmoud², Abdullah Aladwani³, Omar Alnasser⁴, Rayan Alhusennan⁵, Sultan Alotaibi⁶

College of Computer Science and Information Technology, Imam Abdulrahman bin Faisal University, Kingdom of Saudi Arabia

¹2220000240, ²2210002240, ³2220001989, ⁴2220001784, ⁵2220000455, ⁶2220007295 @iau.edu.sa

Abstract

Large Language Models (LLMs) are now widely embedded in software development workflows as code assistants that help developers write, review, and refactor code. Their growing integration into IDEs and continuous-integration pipelines has, however, introduced a new and serious security threat known as prompt injection, in which an attacker crafts malicious input that forces the model to ignore its safety constraints and produce insecure or harmful output or perform unintended actions. This study treats prompt-injection detection as a binary text-classification problem positioned at the perimeter of an LLM-powered code assistant, where each prompt is labelled as benign or injection. Using a 5,000-prompt benchmark derived from a public prompt-injection corpus and split into a stratified 70/30 train–test partition, three classical machine-learning classifiers, namely a Support Vector Machine with an RBF kernel, a Random Forest, and XGBoost, were trained on TF-IDF and Bag-of-Words representations. Conservative text cleaning that preserves injection signatures, inverse-frequency class reweighting, and 5-fold cross-validated grid search were applied, and the models were evaluated using accuracy, precision, recall, F1-score, and ROC-AUC, supported by confusion matrices and feature-importance analysis. The SVM achieved the strongest results, with 89.3% accuracy, an injection-class F1 of 0.869, and a ROC-AUC of 0.960, outperforming the other two classifiers in recall and overall discrimination. The findings show that lightweight lexical detectors can provide a practical and deployable first line of defence for code-assistant environments while remaining transparent and computationally inexpensive.

Keywords: Prompt injection; LLM code assistants; Machine learning; Text classification; TF-IDF; Cybersecurity.

1. Introduction

In just a few short years, large language models (LLMs) have become an essential component of developing software today. As a result, LLM tools like ChatGPT, Claude Code, Gemini Code Assist and GitHub Copilot are being integrated into integrated development environments (IDEs), continuous integration pipelines, and code review environments, where they help generate functions based on natural language descriptions; provide insights into complex code bases; identify bugs; provide suggestions for refactoring code; etc. Based on recent surveys from industry groups, it is clear that a significant portion of professional coders now include AI-assisted coding tools in their day-to-day activities; in fact, many organizations have adopted this type of tool at an enterprise level with the goal of speeding up delivery. These tools clearly help increase productivity; however, they also greatly increase the attack surface for software through methods that traditional security tools were never designed to address.

Unlike traditional software that runs in a predetermined manner based on well-defined inputs, large language models (LLMs) run on natural-language instructions combined with code and the outside world, blurring the line between data and instructions and opening them up to being corrupted through imaginative text.* The main manipulation of LLMs is "prompt injection," where a malicious actor provides new instructions that cause an LLM to act in an unintended manner [3, 4]. Prompt injections can occur in two ways: direct and indirect. A malicious actor can directly inject prompts into an LLM that will result in an LLM's behaviour being altered or indirectly through instructions that have been concealed in external data (i.e., comments in code, documentation, issue trackers, pull requests, or web pages) that are later read by the LLM [2, 4]. The unintended consequences of prompt injections on software applications have been grave; these have ranged from the hard-coded insertion of backdoors and credentials to the extraction of keys for API access and proprietary source code, and in the case of LLM being used as agents, through the unauthorised modification or execution of commands against files.

Many researchers have found that, although many of the best possible LLM models employ safeguards, they still remain extremely susceptible to attack (with reported independent penetration test success rates greater than 80% on LLM integrated applications). For example, 31 out of 36 LLM integrated commercial applications were vulnerable according to tests run using HouYi [3]. Similarly, prompt injection vulnerabilities were shown to exist across all models and tasks in recent benchmark studies [2]. Thus, traditional defenses (i.e., keyword filtering, static rule sets) are not effective against prompt injection because they rely on the heuristics (i.e., rules of thumb) rather than the semantic meaning behind the malicious prompt (i.e., there is no known rule that can be used predictively). In contrast, machine learning could provide a better method for detecting prompt injections, i.e., by allowing classifiers to learn to recognize malicious prompts based on semantics rather than being dependent on weak heuristics [5][11].

While there has been some advancement, there remains significant gaps in research. Most importantly, there is a lack of a practical and deployable defense against external prompt-injection attacks that can be applied without requiring white-box access to the underlying model, generalize across multiple types of prompts and maintain a low false positive rate such that legitimate developer prompts are not blocked [5], [13], [14]. This research aims to fill this gap by comparing three classical machine-learning classifier types (i.e., Support Vector Machine, Random Forest

and XGBoost) using TF-IDF and Bag-of-Words features on 5,000 prompt test bench data sets. The anticipated outcome of the research is a precise, reproducible, low computation detection pipeline and pragmatic security recommendations for teams who implement LLM-based code assistant solutions.

The remaining part of this work is organized as follows. Section 2 contains a review of related literature. Section 3 contains the proposed machine learning techniques used in this study, namely the SVM, Random Forest, and XGBoost classifiers. Section 4 contains the empirical studies, including dataset description, experimental setup, methodology, and the adopted optimization and parameter search strategy. Section 5 presents the results and discussion, while Section 6 contains the conclusion and recommendations emanating from this work.

Contributions of the Study

- A comparative empirical analysis of three distinct machine-learning classifiers (SVM-RBF, Random Forest, and XGBoost) for prompt-injection detection, evaluated on a stratified 70/30 split of a 5,000-prompt public benchmark using both TF-IDF and Bag-of-Words representations.
- A reproducible, white-box-free detection pipeline that uses conservative, signature-preserving preprocessing, inverse-frequency class reweighting, and 5-fold cross-validated grid-search optimisation, with a fixed random seed for full reproducibility.
- An explicit ablation of class-imbalance handling strategies (class reweighting, no balancing, and SMOTE) showing that inverse-frequency reweighting is preferable to synthetic oversampling in high-dimensional sparse TF-IDF space.
- A transparent explainable-AI analysis of the most discriminative lexical features, revealing the injection markers the detectors rely on and the resulting robustness limitations.
- A benchmarking of the proposed detectors against six prior studies with explicit comparability tagging, together with practical security recommendations for organisations deploying LLM code assistants.

2. Review of Related Literatures

The reviewed literature spans three areas: prompt-injection attacks, broader LLM security research, and machine-learning detection models. This section summarises the most relevant studies, the datasets and techniques they used, their best reported results, and their limitations, and positions the present work with respect to them.

2.1 Prompt Injection Attacks

Liu et al. introduced HouYi, a black-box prompt-injection framework inspired by classical web injection that decomposes an attack into a pre-constructed prompt, a context-separating injection segment, and a malicious payload [3]. Applied to 36 commercial LLM-integrated applications, HouYi found 31 to be vulnerable and achieved an overall attack success rate of about 86.1%, enabling outcomes such as unrestricted LLM usage and theft of proprietary application prompts [3]. Greshake et al. formalised indirect prompt injection, in which adversarial instructions are planted in external content likely to be retrieved at inference time, effectively turning retrieval pipelines into a code-execution surface; their case studies against systems such as Bing Chat and code-completion engines demonstrated data theft and arbitrary tool control without any direct user interaction [4]. Yi et al. proposed the BIPIA benchmark, the first standardised benchmark for indirect prompt injection, and showed that LLMs universally fail to distinguish informational context from actionable instructions; their white-box boundary-aware defence reduced attack success to near zero while preserving output quality [2].

2.2 LLM Security Research

In addition to individual attack research works examining LLM security in a systematic way, Yi et al. conducted a survey of jailbreak attacks and their defenses, categorizing them by attacker access and using gradient optimization, logit manipulation, fine tuning, and in-context adversarial as the categories of attack. They also reported that most existing defenses against LLM security lack robustness against adaptive adversaries. Qi et al. demonstrated that the level of safety alignment in many LLMs is limited and often only occurs within the first few output tokens, meaning that once the first refusal is successfully bypassed, no further refusals will be made by the model. To address the narrowness of the alignment, Qi proposed the use of data augmentation techniques and constrained fine-tuning to enhance the length of LLM safety alignment. Bhatt et al. developed CyberSecEval 2, a comprehensive suite for evaluating the security of LLM models related to secure code development, determining whether or not there are vulnerabilities, resisting prompt injection attacks, and whether code interpreters can be abused through prompt injection; CyberSecEval 2 produced measurable prompt injection success data using advanced models such as GPT-4 and LLaMA 3 as well as established the existence of safety and utility trade-offs between prompts. Mathew provided a detailed review that categorized injection attack techniques into direct, indirect and multi-step injection and reviewed both preventative and detection based defenses against those attacks and found that using prompt level defense only were not sufficient to secure LLM systems. Fu et al. created PoisonBench to show how even small numbers of poisoned samples can significantly impact the behavior of LLMs while retaining benchmark performance level ratings, thus raising concerns about supply chain risk associated with using public LLM model repositories.

2.3 Machine-Learning Detection Models

Detection-oriented studies fall into external and internal approaches. Jain et al. compared classical and deep-learning classifiers with multiple vectorisations (TF-IDF, Bag-of-Words, Word2Vec, embeddings) on a Hugging Face prompt-injection dataset, with a recurrent neural network achieving the best detection rate of 94.74% [5]. Ayub and Majumdar used embedding-based classifiers (OpenAI, GTE, MiniLM) combined with Random Forest and Logistic Regression, reaching an AUC of 76.4% and an F1 of 86.8%, but reported difficulty detecting novel attacks [11]. Rahman et al. applied multilingual BERT embeddings with classical classifiers and obtained 96.5% average accuracy with 100% precision on a single dataset [12], while Ivry and Nahum fine-tuned ModernBERT and reported 98.7% average accuracy and 98% F1, though retraining is required for novel injections [13]. Internal-behaviour methods include the Attention Tracker of Hung et al., which inspects attention heads and reaches up to 100% AUROC without training [14], and the hidden-state and gradient-feature approach of Wen et al., which fuses features inside the model and achieves 98% accuracy [15]; both, however, require white-box access and incur high computational overhead.

Overall, the literature confirms that prompt injection is a practical and universal threat, that contextual embedding and white-box methods achieve the highest accuracy, and that external classical detectors offer a deployable but lexically dependent alternative. The present work is distinguished by focusing on lightweight, white-box-free classical detectors evaluated transparently on a single consistent benchmark, directly addressing the deployability and low-false-positive requirements identified as the research gap.

Table 1: Summary of previous studies reviewed in this work.

Study / Ref.	Dataset	No. of Samples	No. of Features	Best Technique	Best Result
Liu et al. [3]	36 commercial LLM apps	36 apps	Not specified	HouYi black-box injection	86.1% attack success; 31/36 vulnerable
Greshake et al. [4]	Real systems (Bing Chat, code engines)	Not specified	Not specified	Indirect injection taxonomy	Demonstrated data theft & tool control
Yi et al. [2]	BIPIA benchmark	Not specified	Not specified	Boundary awareness (white-box)	Attack success $\approx$ 0%
Yi et al. [6]	Literature survey	Not specified	Not specified	Jailbreak taxonomy/survey	Qualitative (no single metric)
Qi et al. [7]	Open-weight LLMs (LLaMA)	Not specified	Not specified	Shallow-alignment analysis	Alignment concentrated in first tokens
Bhatt et al. [8]	CyberSecEval 2	Not specified	Multiple tasks	Benchmark evaluation	Injection success across models
Mathew [9]	Review (AdvBench, MS MARCO)	Not specified	Not specified	Review of attacks/defenses	Defenses only partially effective
Fu et al. [10]	PoisonBench	Not specified	Not specified	Data-poisoning benchmark	Small poison strongly alters behavior
Jain et al. [5]	HF prompt-injection dataset	Not specified	Not specified	RNN + NLP vectorizers	94.74% detection rate
Ayub & Majumdar [11]	Open-source prompts (80/20)	1539 / 1027 / 387	Embedding dims	OpenAI embeddings + RF	AUC 76.4%, F1 86.8%
Rahman et al. [12]	Deepset PI DB (80/20)	Not specified	Not specified	Multilingual BERT + LR/SVM/RF	96.5% acc, 100% precision
Ivry & Nahum [13]	Open benign + jailbreak (90/10)	Not specified	Not specified	ModernBERT (fine-tuned)	98.7% acc, 98% F1
Hung et al. [14]	Open-PI + Deepset, 5 LLMs	Not specified	Attention heads	Attention Tracker (white-box)	Up to 100% AUROC
Wen et al. [15]	Wiki/News, LaMini, BIPIA	Not specified	Hidden-state + gradient	MLP on fused internals (white-box)	98% acc, 0.7% ASR

3. Material & Methods

Prompt-injection detection is treated as a binary text-classification problem situated at the perimeter of an LLM-powered code assistant. A user prompt is classified as either benign ($y = 0$), a legitimate developer query such as a code-completion or refactoring request, or injection ($y = 1$, also denoted jailbreak), any input whose intent is to override the system prompt, exfiltrate data, jailbreak alignment, or manipulate downstream tool calls [3], [4], [9]. The full pipeline comprises ten stages: dataset loading from local CSV files, exploratory data analysis, text preprocessing, feature extraction (TF-IDF and Bag-of-Words), class-imbalance treatment, supervised training of three classifiers (SVM-RBF, Random Forest, XGBoost), 5-fold cross-validated grid-search optimisation, multi-metric evaluation, benchmarking against the literature, and a lightweight explainable-AI step. The implementation is in pure Python using scikit-learn 1.3, xgboost 1.7, imbalanced-learn 0.11, and Hugging Face datasets 2.14, with a fixed random seed of 42 for reproducibility [20].

3.1 Description of the Dataset

The experimental data were generated from a public prompt-injection benchmark hosted on Hugging Face (rogue-security/prompt-injections-benchmark, CC-BY-NC-4.0), which is the migrated version of the QualiFire benchmark originally referenced in the proposal [19]. The corpus contains 5,000 prompts in plain text, predominantly English with a substantial German subset, each row consisting of a text column (the raw prompt) and a label in {benign, jailbreak}. The data were saved as a stratified 70/30 train–test pair: a training file of 3,500 prompts (2,098 benign, 1,402 jailbreak) and a held-out test file of 1,500 prompts (899 benign, 601 jailbreak). The target variable is therefore the binary label distinguishing benign developer prompts from injection prompts, making this a classification task with two categories. The split is computed in advance and used in every experiment; the training file is used exclusively for fitting and 5-fold cross-validation, while the test file is consumed once, after hyperparameter selection, to estimate generalisation.

Table 2: Dataset summary.

Property	Training set	Test set
Total prompts	3,500	1,500
Benign (label 0)	2,098	899
Injection (label 1)	1,402	601
Class ratio (benign:injection)	1.50 : 1	1.50 : 1
Features	text, label	text, label

The methodology workflow that links these stages is summarised in Figure 1.

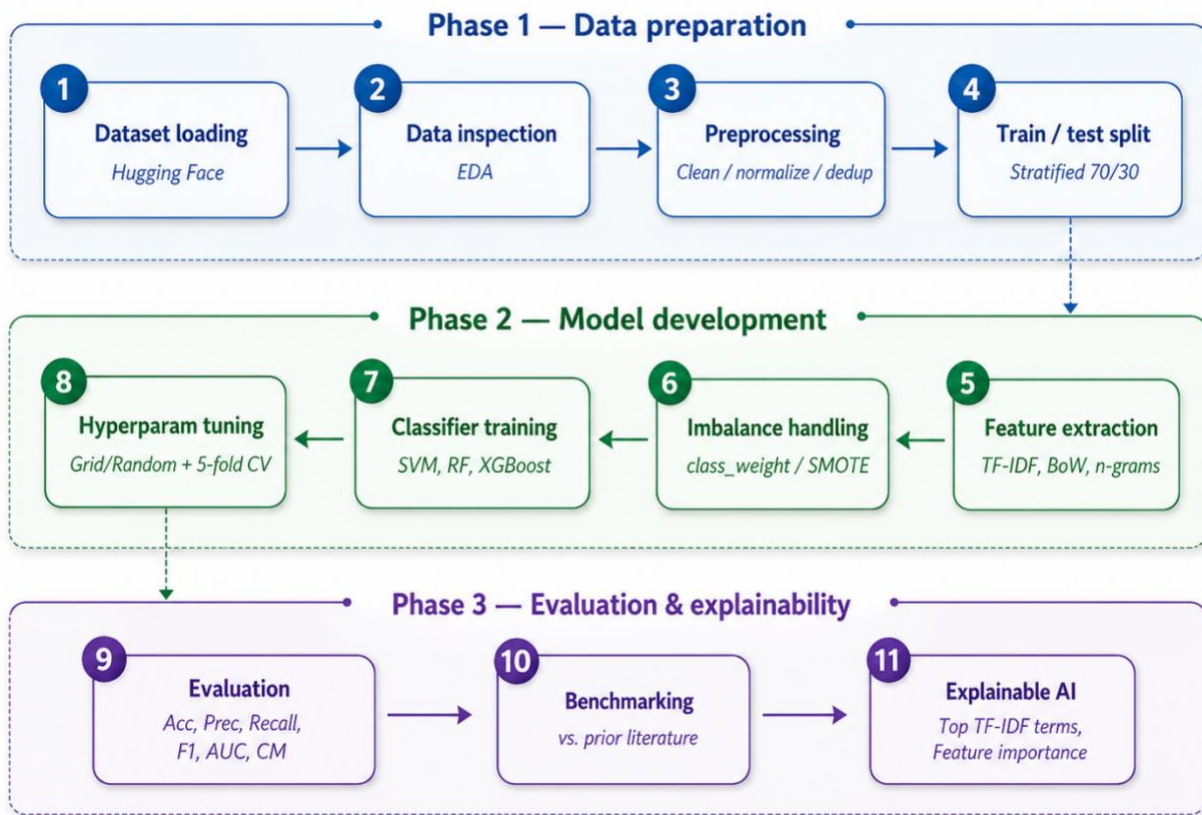


Figure 1: Methodology workflow of the proposed machine learning approach.

3.1.1 Statistical Analysis of the Dataset

Because the input is raw text rather than numerical attributes, statistical analysis focuses on prompt length (in characters) and on the class distribution. The length statistics over the full corpus are a median of 87, a mean of 432, a standard deviation of 1,014, a minimum of 1, and a maximum of approximately 31,000 characters. The heavy right-tail is driven by indirect-injection prompts that paste long external context [4]. Table 3 summarises these statistics, and Figure 3 shows the per-class length distribution, which confirms that injection prompts skew longer than benign ones.

Table 3: Statistical analysis of the dataset (prompt length in characters).

Feature	Mean	Median	Std. Dev.	Maximum	Minimum
Prompt length (chars)	432	87	1,014	≈31,000	1

The class distribution of the unified corpus is shown in Figure 2. The training set is mildly imbalanced at 59.9% benign and 40.1% injection (a ratio of 1.50 : 1), and the test set inherits this ratio by construction. The imbalance is mild but is treated explicitly, as discussed in Section 4.2.



Figure 2: Number of records per class category in the dataset (benign vs. injection).

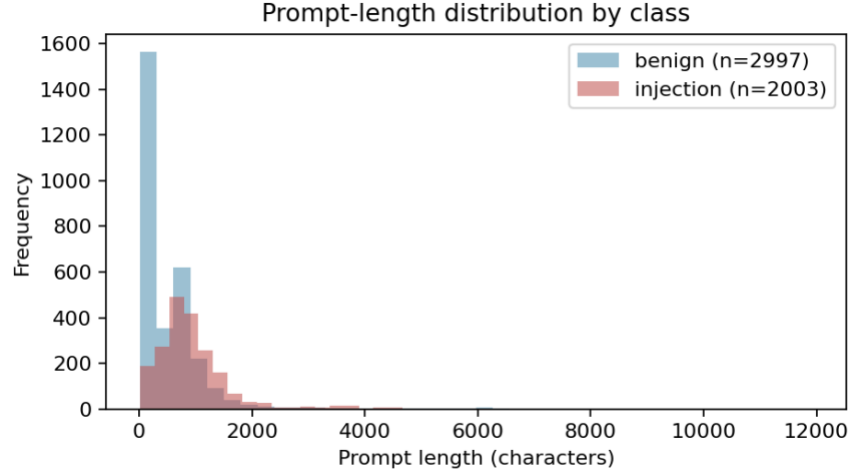


Figure 3: Prompt-length distribution by class (characters).

No synthetic data balancing was retained in the final pipeline; instead, inverse-frequency class reweighting was used (Section 4.2). A before-and-after balancing figure is therefore not applicable to this study. Because the features are derived dynamically from text by the vectoriser, a fixed attribute-to-target correlation table is likewise not applicable; the discriminative power of individual lexical features is instead examined through model-based feature importance in Section 3.3 and Section 5.

3.2 Preprocessing

Prompt-injection signatures rely heavily on functional tokens such as “ignore the above”, “disregard previous”, “you are now”, or the German “vergiss alles” (“forget everything”). Aggressive normalisation would erase the very phrases the detectors must recognise, so a deliberately conservative cleaning regime is adopted, with each step justified below.

- **Unicode normalisation (NFKC):** zero-width characters and homoglyph variants commonly used to bypass keyword filters are removed, and control characters are replaced with whitespace, so that obfuscated payloads are normalised to a canonical form [9].
- **Whitespace normalisation:** runs of whitespace are collapsed to a single space and leading/trailing spaces, tabs, and carriage returns are stripped, ensuring consistent tokenisation.
- **Lowercasing:** applied because TF-IDF and Bag-of-Words are case-sensitive, so that “Ignore” and “ignore” map to a single feature [5], [13].
- **Stop-words are NOT removed:** a deliberate departure from conventional text classification, because injection signatures live in common function words and bigrams such as “ignore the”.
- **Deduplication on (text, label):** prevents metric inflation, a known issue in prompt-injection corpora [5]. After preprocessing the train and test files retain all 3,500 and 1,500 rows respectively.

Because the dataset is pre-labelled, contains no missing values, and is already split, no missing-value imputation, label encoding beyond the binary mapping, or additional train–test splitting was required.

3.3 Feature Selection / Feature Extraction

Two text representations are compared exactly as scoped in the proposal: TF-IDF as the primary representation and Bag-of-Words (BoW) as a baseline. For a term t in document d , $\text{BoW}(t,d) = \text{count}(t,d)$. TF-IDF uses the scikit-learn formulation:

$$\begin{aligned} \text{tf}(t, d) &= \text{count}(t, d) / \sum \text{count}(t', d) \\ \text{idf}(t, D) &= \ln((1 + |D|) / (1 + \text{df}(t))) + 1 \\ \text{tfidf}(t, d) &= \text{tf}(t, d) \cdot \text{idf}(t, D), \text{ followed by L2 normalisation} \end{aligned}$$

Sublinear term-frequency scaling ($1 + \ln(\text{tf})$) is enabled to dampen highly repetitive injection payloads such as long roleplay templates. The vectoriser configuration is summarised in Table 4.

Table 4: TF-IDF / vectoriser parameters and justification.

Parameter	Value	Justification
ngram_range	(1, 2)	Captures phrases like “ignore previous” and “you are now” [5].
min_df	2	Drops once-only typos and rare URL fragments.
max_df	0.95	Drops terms in >95% of documents (effective stop-word filter).
max_features	20,000	Caps dimensionality.
strip_accents	unicode	Normalises diacritics (e.g., German prompts).

No explicit chi-squared or mutual-information feature selection is applied, for two reasons: TF-IDF with $\text{min_df} = 2$ and $\text{max_df} = 0.95$ already prunes pathological features, and the tree-based classifiers (Random Forest and XGBoost) perform embedded selection through impurity-based splits. Post-hoc feature importance is instead inspected for transparency; Figure 4 shows the top-25 discriminative features by Random Forest impurity importance, dominated by injection markers such as “ignore”, “previous”, and “instructions”.

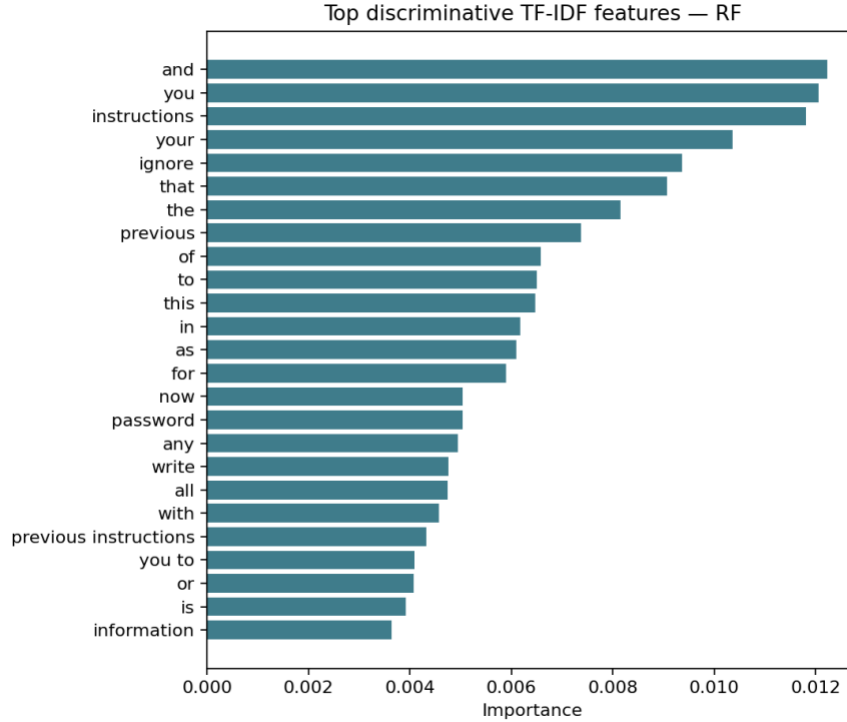


Figure 4: Top-25 discriminative TF-IDF features — Random Forest impurity importance.

3.4 Description of the Classifiers

Three classifiers are trained, each wrapped in a scikit-learn pipeline of the form Vectoriser → (optional sampler) → Classifier to keep fit/predict semantics identical and prevent test-time leakage from the vectoriser.

3.4.1 Classifier #1: Support Vector Machine (RBF kernel)

A Support Vector Machine (SVM) separates the two classes by a maximum-margin hyperplane in a possibly infinite-dimensional feature space. The soft-margin primal formulation is:

$$\min \frac{1}{2} \|w\|^2 + C \cdot \sum_i \xi_i \quad \text{subject to} \quad y_i(\langle w, \phi(x_i) \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

where $C > 0$ trades margin width for slack. With the RBF kernel,

$$K(x_i, x_j) = \exp(-\gamma \cdot \|x_i - x_j\|^2)$$

non-linear decision surfaces are realised directly in the sparse TF-IDF space without explicit feature mapping. SVM-RBF is well suited to prompt-injection text because it generalises well in high-dimensional sparse settings and its kernelised distance metric captures phrase-level similarity that purely lexical models miss. The `class_weight = 'balanced'` option rescales the per-class slack penalty by inverse class frequency.

3.4.2 Classifier #2: Random Forest

A Random Forest is an ensemble of decorrelated decision trees [17]. Given a training set, B trees are fitted to bootstrap samples, and at each split a random subset of features is considered. The ensemble prediction is the modal class:

$$\hat{y}(x) = \text{mode}\{T_1(x), T_2(x), \dots, T_B(x)\}$$

Each split maximises the reduction in Gini impurity:

$$\text{Gini}(\text{node}) = 1 - \sum_k p_k^2, \quad \Delta \text{Gini} = \text{Gini}(\text{parent}) - \sum_c (n_c / n) \cdot \text{Gini}(c)$$

Random Forests suit high-dimensional sparse text because bagging and feature subsampling reduce variance, impurity-based splits naturally handle correlated bigrams, and the impurity scores yield directly interpretable feature importances — a cheap form of explainable AI.

3.4.3 Classifier #3: XGBoost (Extreme Gradient Boosting)

XGBoost minimises a regularised additive objective over an ensemble of regression trees [16]:

$$L(\Phi) = \sum_i \ell(y_i, \hat{y}_i) + \sum_t \Omega(f_t), \quad \Omega(f) = \gamma \cdot T_f + \frac{1}{2} \cdot \lambda \cdot \|w\|^2$$

Using a second-order Taylor expansion with gradients g_i and Hessians h_i , the optimal leaf weight in leaf j and the split gain are:

$$w_{j^*} = - \sum g_i / (\sum h_i + \lambda)$$

$$G_{\text{split}} = \frac{1}{2} [(\sum_L g)^2 / (\sum_L h + \lambda) + (\sum_R g)^2 / (\sum_R h + \lambda) - (\sum g)^2 / (\sum h + \lambda)] - \gamma$$

XGBoost is included because it routinely leads structured-tabular benchmarks and is a strong baseline for sparse text features, and its `scale_pos_weight` parameter offers a distinct treatment of class imbalance. The histogram tree method (`tree_method = 'hist'`) is used for fast training.

4. Experimental Setup

All experiments were carried out in Python using scikit-learn 1.3 for the SVM and Random Forest, xgboost 1.7 for XGBoost, imbalanced-learn 0.11 for the SMOTE ablation, and the Hugging Face datasets 2.14 library for data loading. A fixed random seed of 42 was propagated through every NumPy, scikit-learn, and XGBoost call so that the results are fully reproducible; the implementation and project files are publicly available in the PIShield GitHub repository [20]. The dataset was partitioned using the predefined stratified 70/30 split: the 3,500-prompt training file was used for fitting and 5-fold stratified cross-validation, and the 1,500-prompt held-out test file was used once, after hyperparameter selection, to estimate generalisation. Each classifier was implemented as a single pipeline (Vectoriser → optional sampler → Classifier) to guarantee that the vectoriser is fitted only on training folds. The evaluation metrics, appropriate for this binary classification task, are accuracy, precision, recall, F1-score, macro-F1, weighted-F1, and ROC-AUC, complemented by full confusion matrices and ROC curves; they are defined as follows.

$$\begin{aligned} \text{Accuracy} &= (TP + TN) / (TP + TN + FP + FN) \\ \text{Precision} &= TP / (TP + FP) \quad \text{Recall} = TP / (TP + FN) \\ \text{F1} &= 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall}) \end{aligned}$$

4.1 Optimization Strategy

Hyperparameter optimisation was performed with grid search over a compact, defensible search space using 5-fold stratified cross-validation on the training set, scored by the F1-score of the injection class, which is the operationally meaningful metric because it balances false negatives that let attacks through against false positives that block legitimate developer prompts. The search grids, best parameters, and cross-validated F1 scores are reported in Table 5.

Table 5: Optimum parameters for the proposed models.

Model	Parameter	Values Tested	Optimal Value	CV F1
SVM-RBF	C	{0.5, 1, 2, 4}	4.0	0.8558
	γ	{scale, 0.1, 0.5}	0.5	
Random Forest	n_estimators	{200, 400}	400	0.8002
	max_depth	{None, 20, 40}	None	
	min_samples_split	{2, 5}	5	
XGBoost	n_estimators	{200, 400}	400	0.8059
	max_depth	{4, 6, 8}	6	
	learning_rate	{0.05, 0.1, 0.2}	0.1	

Tuning produced modest gains over sensible defaults (SVM 0.8440 → 0.8558, Random Forest 0.7895 → 0.8002, XGBoost unchanged at 0.8059), which is expected given the deliberately compact grid and the size of the training corpus; further refinement is unlikely to change the qualitative ordering of the classifiers.

4.2 Imbalanced-Data Handling Strategy

The corpus is mildly imbalanced (1.50 : 1 benign-to-injection). An explicit ablation compared three strategies under identical 5-fold cross-validation: inverse-frequency class reweighting, no balancing, and SMOTE oversampling (Table 6). Because the absolute differences are small (within about ± 2 percentage points of F1) and SMOTE interpolates synthetic vectors that have no corresponding natural-language prompt in the 20,000-dimensional TF-IDF space, inverse-frequency reweighting (class_weight = 'balanced' for SVM and Random Forest, scale_pos_weight for XGBoost) was adopted: it modifies the loss without inventing data, preserves the test-time decision rule, and adds zero computation.

Table 6: Imbalance-handling strategy comparison (5-fold CV on the training file).

Strategy	Model	CV F1 (mean)	F1 std	Recall	Prec.
class_weight	SVM	0.8558	0.0147	0.8602	0.8517
class_weight	RF	0.7813	0.0119	0.8231	0.7436
class_weight	XGB	0.8007	0.0097	0.8281	0.7751
no_balance	SVM	0.8558	0.0147	0.8602	0.8517
no_balance	RF	0.7877	0.0144	0.7974	0.7785
no_balance	XGB	0.7890	0.0032	0.7732	0.8057
smote	SVM	0.8565	0.0149	0.8609	0.8524
smote	RF	0.7874	0.0067	0.8531	0.7311
smote	XGB	0.7843	0.0083	0.7817	0.7872

5. Results and Discussion

The three optimised classifiers were evaluated on the held-out test set of 1,500 prompts. Table 7 reports the full set of metrics, and Figure 5 visualises the headline scores. The SVM-RBF detector is the strongest overall, achieving the highest accuracy (0.8933), the highest recall (0.8819), the best injection-class F1 (0.8689), and the highest ROC-AUC (0.9601). XGBoost is the most balanced classifier, with precision and recall close together (0.8202 / 0.8502) and the fastest inference time, making it attractive where the precision–recall trade-off must be tuned via the decision threshold. Random Forest is the cheapest to train (about 6 seconds) but trades precision for recall (0.7572 / 0.8769), producing more false alarms.

Table 7: Held-out test metrics for the three optimised classifiers (TF-IDF).

Model	Accuracy	Precision	Recall	F1	Macro-F1	ROC-AUC
SVM	0.8933	0.8562	0.8819	0.8689	0.8895	0.9601
RF	0.8380	0.7572	0.8769	0.8126	0.8350	0.9315
XGB	0.8653	0.8202	0.8502	0.8350	0.8606	0.9433

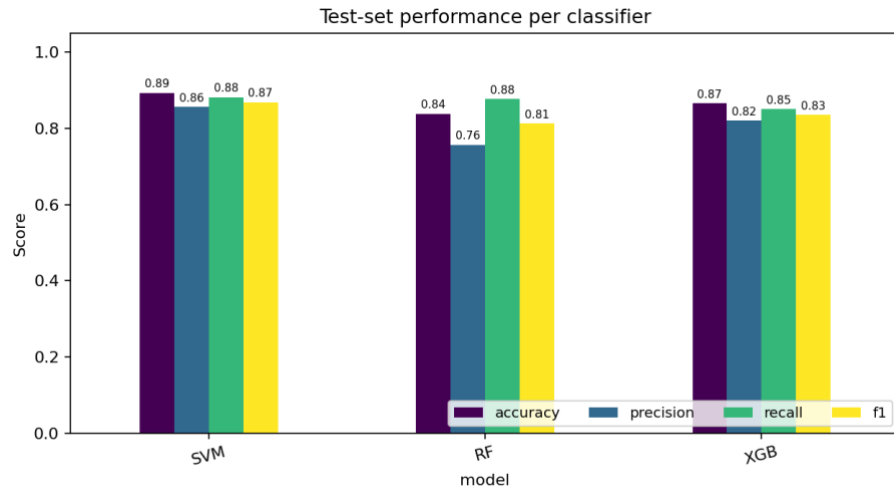


Figure 5: Per-classifier metric comparison (Accuracy, Precision, Recall, F1).

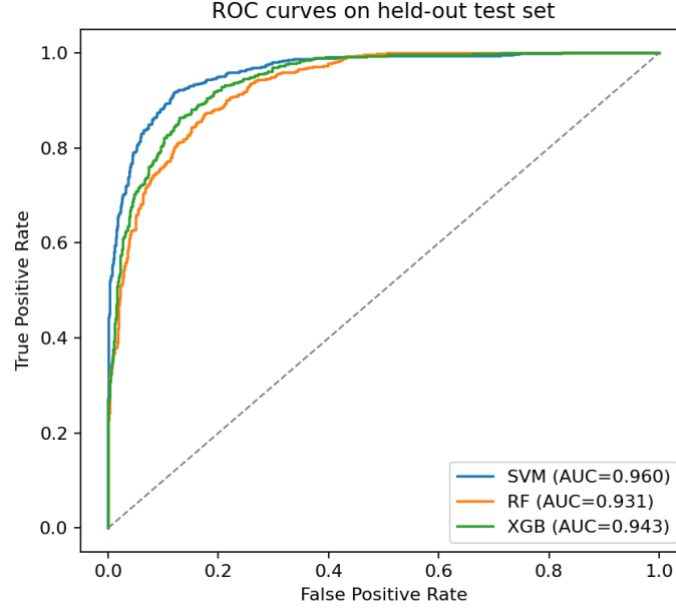


Figure 6: ROC curves on the held-out test set.

The ROC curves in Figure 6 confirm the ranking, with the SVM curve dominating across most of the false-positive range (AUC 0.960), followed by XGBoost (0.943) and Random Forest (0.931). The confusion matrices in Figures 7a–7c provide the error breakdown. The SVM misclassifies 89 benign prompts and misses 71 injections; Random Forest misses fewer injections in relative terms but flags 169 benign prompts as malicious; XGBoost achieves the most even error profile (112 false positives, 90 false negatives).

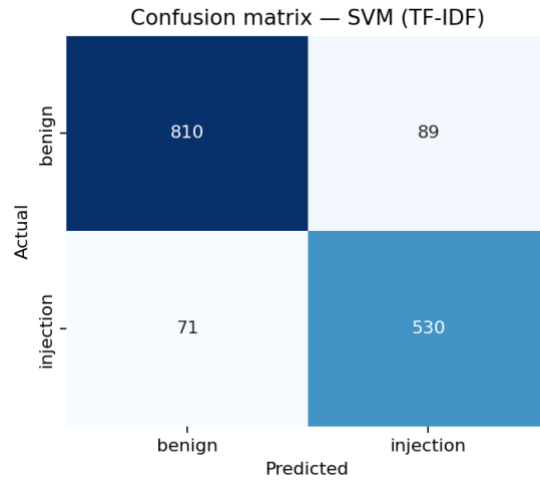


Figure 7a: Confusion matrix — SVM (TF-IDF).

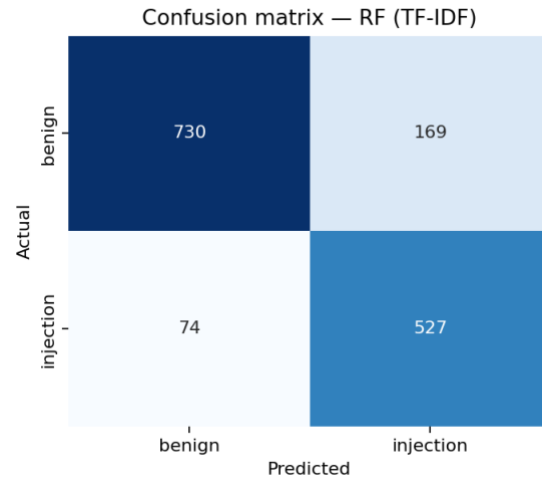


Figure 7b: Confusion matrix — Random Forest (TF-IDF).

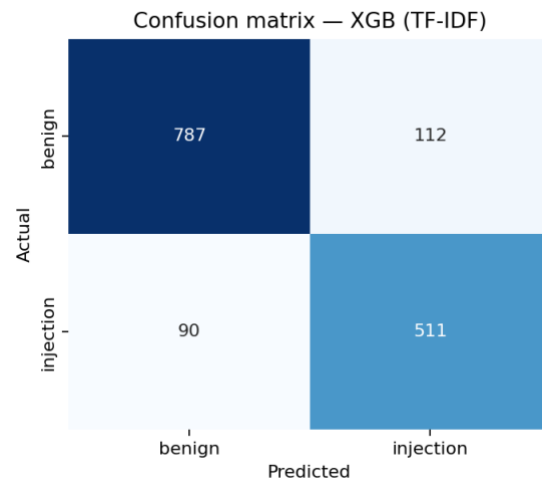


Figure 7c: Confusion matrix — XGBoost (TF-IDF).

5.1 Results of Investigating the Effect of Feature Representation

Because the present study compares feature representations rather than applying recursive feature elimination, the feature-effect analysis contrasts TF-IDF (primary) with Bag-of-Words (baseline) for all three classifiers under default settings on the held-out test set, with 5-fold cross-validation used during model selection on the training file. Table 8 reports the comparison. TF-IDF yields the best single result overall through the SVM (F1 0.8562 with default settings, rising to 0.8689 after tuning), while Bag-of-Words is competitive for the tree-based models but slightly weaker for the SVM. The additional inverse-document-frequency weighting in TF-IDF therefore provides a meaningful benefit for the margin-based SVM, which is the configuration carried forward to the final model.

Table 8: Results of different feature subsets (TF-IDF vs. Bag-of-Words, F1-score on the held-out test set).

Feature Representation	SVM	Random Forest	XGBoost
TF-IDF (all features)	0.8562	0.8180	0.8293
Bag-of-Words (baseline)	0.8028	0.8163	0.8304
Best result	0.8562 (TF-IDF)	0.8180 (TF-IDF)	0.8304 (BoW)

On the basis of these results, the final model was built using the SVM-RBF classifier with TF-IDF features (the best-performing representation) and the optimal hyperparameters from Table 5 ($C = 4.0$, $\gamma = 0.5$). Its held-out performance is summarised in Table 9; the corresponding confusion matrix and ROC curve are Figures 7a and 6 respectively.

Table 9: Results of using the best options based on the optimal parameters obtained (final SVM-RBF model).

Metric	Final SVM (TF-IDF)
Accuracy	0.8933
Precision	0.8562
Recall	0.8819
F1-score	0.8689
ROC-AUC	0.9601

5.2 Comparison of the Proposed Model with Benchmark Studies

Table 10 places the three proposed classifiers next to six prior studies from the literature review. Because the prior works use different prompt-injection corpora, each comparison is tagged by its level of comparability rather than claimed as an exact head-to-head benchmark. The proposed SVM is competitive in the same recall band as Jain et al. [5] (0.8819 vs. 0.9474) and matches the F1 of the embedding-plus-classifier pipeline of Ayub and Majumdar [11] (0.8689 vs. 0.8680). The contextual BERT-family detectors [12], [13] and the white-box attention and hidden-state detectors [14], [15] report higher F1 scores, but this gap is structural: semantic embeddings and access to LLM internals capture meaning that bag-of-n-grams cannot, and the white-box methods require internal access that the present threat model deliberately excludes. The proposed pipeline therefore trades a few points of accuracy for deployability, transparency, and very low computational cost.

Table 10: Benchmarking against prior literature, with comparability tags.

Study / Ref.	No. Feat.	Best Technique	Accuracy	Recall	AUC
Jain et al. [5]	N/S	RNN + vectorizers	–	0.9474	–
Ayub & Majumdar [11]	Embed.	OpenAI embed. + RF/LR	0.8670	0.8680	0.7640
Rahman et al. [12]	N/S	Multilingual BERT	0.9650	0.9330	0.9650
Ivry & Nahum [13]	N/S	ModernBERT (fine-tuned)	0.9870	0.9910	0.9800
Hung et al. [14]	Attn.	Attention Tracker (WB)	–	–	1.0000
Wen et al. [15]	Internals	Hidden-state + grad. (WB)	0.9800	–	–
Proposed — SVM	TF-IDF	SVM-RBF + TF-IDF	0.8933	0.8819	0.9601
Proposed — RF	TF-IDF	RF + TF-IDF	0.8380	0.8769	0.9315
Proposed — XGB	TF-IDF	XGBoost + TF-IDF	0.8653	0.8502	0.9433

5.3 Explainable AI

Two cheap, post-hoc explainability techniques exposed by the tree-based classifiers were applied. The Random-Forest impurity importances (Figure 4) are dominated by linguistic injection markers such as “ignore”, “previous”, and “instructions”, German equivalents, and pronoun anchors characteristic of role-override prompts. The XGBoost gain importances (Figure 8) rank features such as “ignore”, “you need”, “instructions”, and “output format” highly, consistent with the imperative-reframing patterns documented in HouYi [3] and BIPIA [2]. Both rankings are dominated by function words and short phrases, which validates the use of low-order n-grams over heavy preprocessing but also explains the classifiers’ residual errors: detection relies on lexical cues that an adaptive adversary could paraphrase.

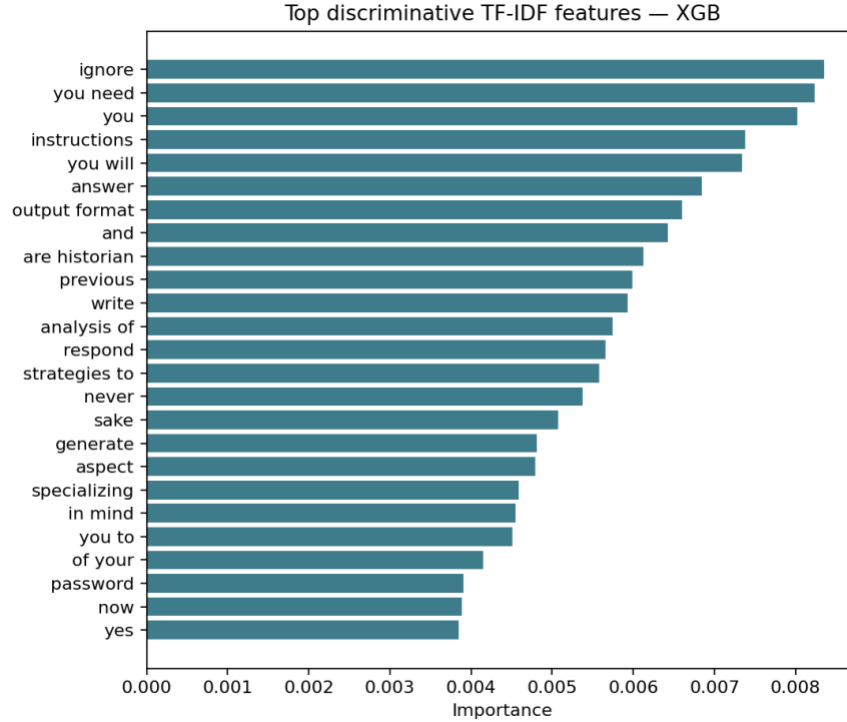


Figure 8: Top-25 discriminative TF-IDF features — XGBoost gain importance.

Limitations of the Current Study

- The detectors rely on lexical (bag-of-n-grams) features, so an adaptive adversary who paraphrases injection markers or uses novel phrasing could evade detection, as the feature-importance analysis indicates.
- Evaluation uses a single benchmark corpus of 5,000 prompts; although it includes English and German, generalisation to other languages, domains, and emerging attack styles was not externally validated.
- The study is limited to three classical classifiers and two feature representations; contextual embedding models (e.g., BERT, ModernBERT) and deep architectures were not implemented.
- Only prompt-level detection is addressed; the pipeline was not deployed and tested inside a live code-assistant or IDE, so real-world latency and integration effects remain unmeasured.
- The corpus is mildly imbalanced, and although this was treated by reweighting, the absolute number of injection examples constrains how rare or sophisticated attack variants are represented.

6. Conclusion and Recommendation

This study addressed the growing security threat that prompt-injection attacks pose to LLM-powered code assistants, where adversaries craft malicious prompts that cause the model to ignore safety constraints and produce insecure or harmful output. The problem was framed as a binary text-classification task at the perimeter of the assistant, and three classical machine-learning classifiers, namely an SVM with an RBF kernel, a Random Forest, and XGBoost, were trained and compared on a 5,000-prompt public benchmark using TF-IDF and Bag-of-Words features, conservative signature-preserving preprocessing, inverse-frequency class reweighting, and 5-fold cross-validated grid search.

The SVM-RBF detector with TF-IDF features emerged as the best-performing model, achieving 89.3% accuracy, an injection-class F1 of 0.869, a recall of 0.882, and a ROC-AUC of 0.960 on the held-out test set, outperforming both tree-based classifiers in overall discrimination while remaining transparent and computationally inexpensive. Benchmarking showed that the pipeline is competitive with comparable external detectors and trades a modest amount of accuracy, relative to contextual-embedding and white-box methods, for deployability without access to model internals. These results demonstrate that a lightweight lexical detector can serve as a practical first line of defence for code-assistant environments.

For future work, the following directions are recommended:

- Incorporate contextual embedding models such as BERT and ModernBERT, and explore deep architectures, to capture semantic patterns beyond lexical cues and improve robustness to paraphrased attacks.
- Apply adversarial data augmentation and evaluate against adaptive adversaries to strengthen generalisation to unseen injection strategies.
- Expand evaluation to larger and more diverse multilingual datasets and additional standardised benchmarks for external validation.
- Deploy and evaluate the detection pipeline within a live IDE or code-assistant environment to measure real-world latency, integration cost, and false-positive impact on developers.

Acknowledgement

The authors gratefully acknowledge the guidance of Dr. Yasir Alguwaifli and the support of the College of Computer Science and Information Technology at Imam Abdulrahman bin Faisal University. We also thank our team members for their collaboration throughout this project.

References

- [1] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending Against Prompt Injection with Structured Queries," arXiv preprint arXiv:2402.19643, Feb. 2024.
- [2] J. Yi, Y. Xie, B. Zhu, K. Hines, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and Defending against Indirect Prompt Injection Attacks on Large Language Models," in Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD), Aug. 2024.
- [3] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, Y. Zheng, and Y. Liu, "Prompt Injection Attack Against LLM-Integrated Applications," arXiv preprint arXiv:2306.05499, Jun. 2023.
- [4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec), Nov. 2023.
- [5] B. Jain, P. Pawar, D. Gada, T. Patwa, P. Kanani, D. Patil, and L. Kurup, "Comprehensive Analysis of Machine Learning and Deep Learning Models on Prompt Injection Classification Using Natural Language Processing Techniques," Int. Res. J. Multidiscip. Technovation, vol. 7, no. 1, pp. 1–14, Feb. 2025.
- [6] S. Yi, Y. Liu, Z. Sun, T. Cong, X. He, J. Song, K. Xu, and Q. Li, "Jailbreak Attacks and Defenses Against Large Language Models: A Survey," arXiv preprint arXiv:2407.04295, Jul. 2024.
- [7] X. Qi, A. Panda, K. Lyu, X. Ma, S. Roy, A. Beirami, P. Mittal, and P. Henderson, "Safety Alignment Should Be Made More Than Just a Few Tokens Deep," arXiv preprint arXiv:2406.05946, Jun. 2024.
- [8] M. Bhatt et al., "CyberSecEval 2: A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models," arXiv preprint arXiv:2404.13161, Apr. 2024.
- [9] E. S. Mathew, "Enhancing Security in Large Language Models: A Comprehensive Review of Prompt Injection Attacks and Defenses," J. Artif. Intell., vol. 7, pp. 347–363, 2025.
- [10] T. Fu, M. Sharma, P. Torr, S. B. Cohen, D. Krueger, and F. Barez, "PoisonBench: Assessing Large Language Model Vulnerability to Data Poisoning," arXiv preprint arXiv:2410.08811, Oct. 2024.
- [11] M. Ayub and S. Majumdar, "Embedding-Based Classifiers Can Detect Prompt Injection Attacks," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), pp. 257–268, 2024.
- [12] M. Rahman, H. Shahriar, F. Wu, and A. Cuzzocrea, "Applying Pre-Trained Multilingual BERT in Embeddings for Improved Malicious Prompt Injection Attacks Detection," in Proc. 2024 2nd Int. Conf. Artif. Intell., Blockchain, Internet Things (AIBThings), 2024, pp. 1–7.
- [13] D. Ivry and O. Nahum, "Sentinel: SOTA Model to Protect Against Prompt Injections," arXiv preprint arXiv:2506.05446, Jun. 2025.
- [14] K. Hung, C. Ko, A. Rawat, I. Chung, W. Hsu, and P. Chen, "Attention Tracker: Detecting Prompt Injection Attacks in LLMs," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), pp. 2309–2322, 2024.
- [15] T. Wen, C. Wang, X. Yang, H. Tang, Y. Xie, L. Lyu, Z. Dou, and F. Wu, "Defending against Indirect Prompt Injection by Instruction Detection," in Findings Assoc. Comput. Linguist.: EMNLP 2025, 2025.

- [16] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. ACM SIGKDD, 2016, pp. 785–794.
- [17] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [18] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.
- [19] [Dataset] rogue-security/prompt-injections-benchmark, Hugging Face. Available: <https://huggingface.co/datasets/rogue-security/prompt-injections-benchmark> (accessed May 2026).
- [20] PIShield, GitHub repository. Available: <https://github.com/0x00K1/PIShield> (accessed May 2026).